



EMERITUS

Learn. From the world's best.

Recurrent Neural Network - II



EMERITUS

Learn. From the world's best.

Recap

Sequence Processing with Neural Networks

Handling Time-Dependent Data

Sequential data requires neural networks that can handle temporal patterns and dependencies



- **Sequential Data Challenge:** Traditional feedforward networks struggle with variable-length sequences in text, speech and time series
- **RNN Foundation:** Recurrent Neural Networks use hidden states to remember information across sequence steps
- **Encoder-Decoder:** Maps input sequences to output sequences, used in machine translation and image captioning

Advanced Architectures

Memory Management

LSTM & GRU: Use gating mechanisms to manage information, tackling vanishing gradients and retaining long-term memory

Context Enhancement

Bidirectional RNNs: Read sequences forwards and backwards to capture full context



Standard RNN: Basic recurrence with gradient issues



LSTM/GRU: Gated networks for long-term dependencies



Attention Mechanisms: Focus dynamically on important sequence positions

Word Representations

From Words to Vectors

Dense word vectors let neural networks understand and process text effectively

01
10

One-Hot Encoding: Sparse binary vectors; simple but inefficient for large vocabularies



Count-Based Methods: Use co-occurrence statistics and techniques like SVD to reduce dimensions



Prediction-Based (Word2Vec): Neural models predict context words, learning dense vectors with semantic meaning



Hybrid Approaches (GloVe): Combine global statistics with local context for robust embeddings

Key Insight: Embeddings capture semantic similarity, helping models generalise and improve downstream tasks.

Applications – Captioning



Image Captioning

- **CNN Encoder:** Extracts visual features
- **LSTM Decoder:** Generates natural language descriptions



Video Captioning

- Detects events and uses RNNs to describe actions over time

Evaluation Metrics

BLEU: N-gram precision

METEOR: Alignment-based scoring

ROUGE: Recall-focused evaluation

CIDEr: Consensus-based image description

Beyond Captioning

1. **Visual Question Answering: VQA** CNN encodes images, LSTM processes Natural language questions, fusion networks generate answers
 2. **Open-Ended VQA:** Produces flexible, descriptive answers, not just classifications
 3. **Transformer Revolution:** Use attention for better scalability, parallelisation, and performance
- Shift from RNNs to Transformers enables more efficient training and superior results across applications



Text Classification

Text Classification

Automated Text Categorisation

Problem Statement:

Build a GRU-based model to categorise documents into 20 topics using the 20 Newsgroups dataset. The model learns from text data to classify new documents accurately, with applications in spam filtering, document organisation, topic modelling, and content recommendations

Data Description:

The project uses the 20 Newsgroups dataset, containing about 20,000 labelled documents across 20 categories, making it ideal for supervised text classification tasks

Key Characteristics of the Data:

- Total Documents: ~20,000
- Number of Categories: 20
- Examples: *sci.space*, *comp.graphics*, *rec.sport.hockey*, *talk.politics.mideast*, *alt.atheism*

Text Classification

Data Attributes

- **Text:** Document content used as model input
- **Labels:** Numbers (0–19) showing each document's category

Text Preprocessing

- Remove headers, footers, and quotes
- Apply tokenisation, padding, and sequence conversion before GRU input

Use Case

- Automates categorisation of large text collections
- Applications: customer support, news articles, spam filtering, legal documents
- GRU captures sequential patterns, making it effective for text tasks

Text Classification

```
# Importing necessary libraries  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
from sklearn.datasets import fetch_20newsgroups  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import accuracy_score, classification_report  
from tensorflow.keras.preprocessing.text import Tokenizer  
from tensorflow.keras.preprocessing.sequence import pad_sequences  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Embedding, GRU, Dense, Dropout  
from tensorflow.keras.optimizers import Adam
```

Text Classification

- We load the 20 Newsgroups dataset with documents in 20 categories
- The *fetch_20newsgroups()* function provides the texts and their labels

Text Classification

```
# #### Step 1: Load the Dataset  
# Fetching the 20 Newsgroups dataset using sklearn  
print("Loading the 20 Newsgroups dataset...")  
newsgroups_data = fetch_20newsgroups(subset='all', shuffle=True, remove=('headers',  
'footers', 'quotes'))  
  
# Extracting the text data and their corresponding labels  
texts, labels = newsgroups_data.data, newsgroups_data.target  
categories = newsgroups_data.target_names  
  
# Printing basic information about the dataset  
print(f"Number of classes: {len(categories)}")  
print(f"Sample categories: {categories[:5]}")
```

Text Classification

- Split data: 80% training, 20% testing
- Limit vocabulary to 10,000 words (max_words)
- Standardise sequence length to 200 tokens (max_len)
- Tokeniser maps words to indices, using <OOV> for unknown words
- Convert texts to sequences and pad them to uniform length

Text Classification

Step 2: Data Preprocessing

Splitting the data into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(  
    texts, labels, test_size=0.2, random_state=42, stratify=labels  
)
```

Define maximum number of words to consider and sequence length

max_words = 10000 # We'll consider only the top 10,000 words in the dataset

max_len = 200 # We'll truncate or pad each document to a length of 200 tokens

Tokenize the text data

```
tokenizer = Tokenizer(num_words=max_words, lower=True, oov_token="")
```

```
tokenizer.fit_on_texts(X_train)
```

Convert text to sequences of integers

```
X_train_seq = tokenizer.texts_to_sequences(X_train)
```

```
X_test_seq = tokenizer.texts_to_sequences(X_test)
```

Pad sequences to ensure uniform input length

```
X_train_pad = pad_sequences(X_train_seq, maxlen=max_len, padding='post',  
truncating='post')
```

```
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len, padding='post',  
truncating='post')
```


Text Classification

- Sequential model for simplicity
- Embedding: Maps words to 128-dimensional vectors
- GRU Layers: First with 64 units (return_sequences=True), second with 32 units
- Dropout: 50% to prevent overfitting
- The second GRU layer has 32 units for deeper pattern learning
- Dense Layer: Softmax activation for 20-category classification
- Compilation: Adam optimiser, sparse_categorical_crossentropy loss for multi-class labels

Text Classification

Step 3: Build the GRU Model

```
print("\nBuilding the GRU model...")
```

```
model = Sequential()
```

```
# Embedding layer to convert word indices into dense vectors of fixed size (128)
```

```
model.add(Embedding(input_dim=max_words, output_dim=128, input_length=max_len))
```

```
# Adding the first GRU layer with 64 units and return_sequences=True to pass sequences to the next layer
```

```
model.add(GRU(64, return_sequences=True))
```

```
model.add(Dropout(0.5)) # Dropout layer to reduce overfitting
```

```
# Adding a second GRU layer with 32 units for further sequence learning
```

```
model.add(GRU(32))
```

```
# Output layer with softmax activation for multi-class classification (20 classes)
```

```
model.add(Dense(20, activation='softmax'))
```

```
# Compiling the model with Adam optimizer and sparse categorical crossentropy loss
```

```
model.compile(optimizer=Adam(learning_rate=0.001), loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
# Display the model summary
```

```
model.summary()
```

Text Classification

- We train the model using the fit method
- We specify epochs=5 and a batch_size of 64
- Use 20% of training data for validation (validation_split=0.2)

Text Classification

Step 4: Train the Model

```
print("\nTraining the GRU model...")  
history = model.fit(  
    X_train_pad, y_train,  
    epochs=5,  
    batch_size=64,  
    validation_split=0.2  
)
```

Text Classification

- Use the trained model to predict test data
- `argmax()` selects the class with the highest probability
- Calculate test accuracy and show precision, recall, and F1 scores

Text Classification

Step 5: Evaluate the Model

```
print("\nEvaluating the model on test data...")
```

```
y_pred = model.predict(X_test_pad)
```

```
y_pred_classes = np.argmax(y_pred, axis=1) # Get the predicted class labels
```

Calculate accuracy and print classification report

```
accuracy = accuracy_score(y_test, y_pred_classes)
```

```
print(f"\nTest Accuracy: {accuracy:.4f}")
```

Display detailed classification metrics for each category

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred_classes, target_names=categories))
```

Text Classification

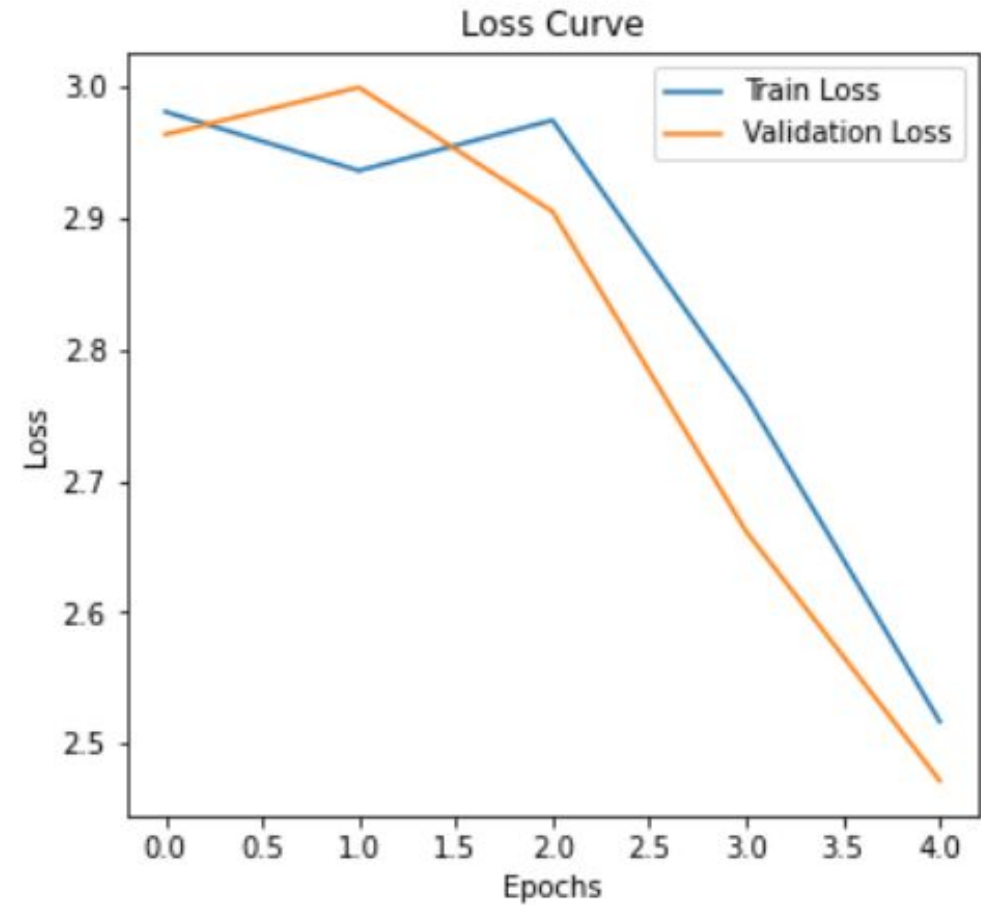
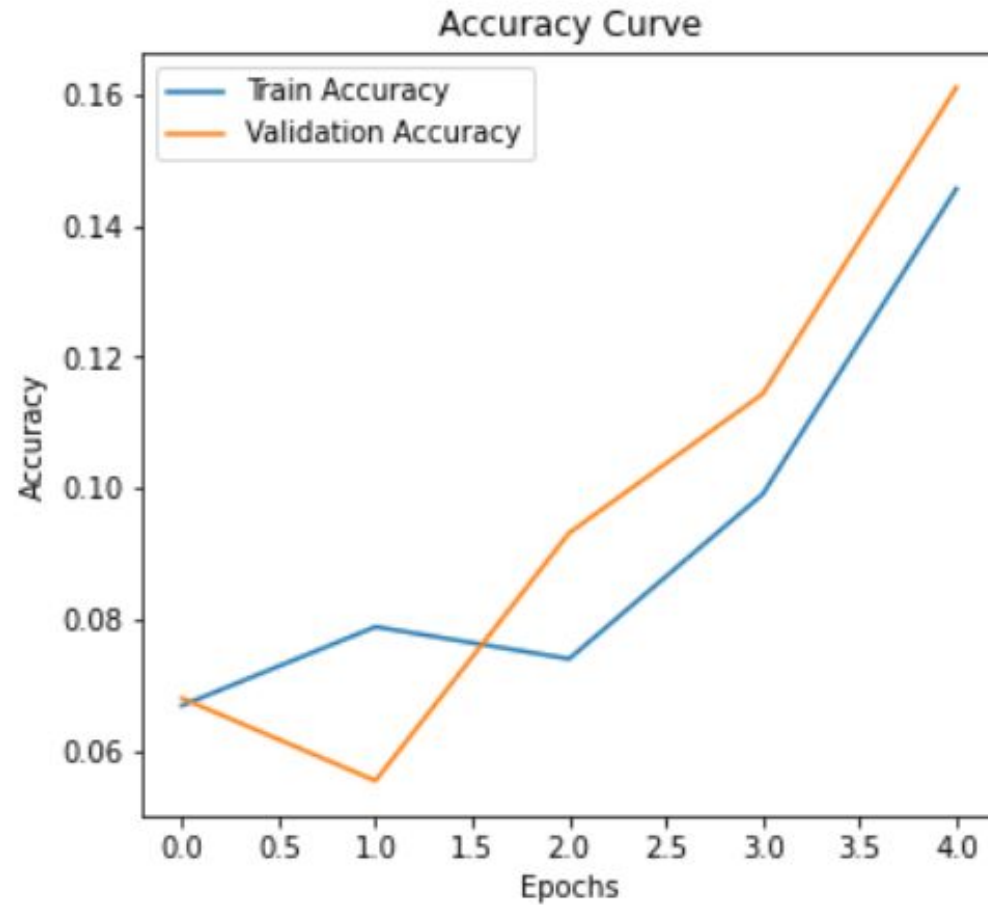
- Plot training and validation accuracy and loss
- Visualise learning progress and detect overfitting
- Summary:

Text Classification

Step 6: Plot Accuracy and Loss Curves

```
plt.figure(figsize=(12, 5))# Plotting training and validation accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy Curve')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()# Plotting training and validation loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss Curve')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```


Text Classification



Q&A Session



Open floor for questions and clarifications.